Topic 1. Problem set — λ-calculus

Mariia Chegodaeva B24-SD-02
ma.chegodaeva@innopolis.university

Problems 1.1 to 1.4 are **[No AI]**. Problem 1.5 marks each of its parts separately. At a defense you may be asked to redo any derivation from this sheet on paper.

Problem 1.1 (α-equivalence) [No AI]

Twelve terms:

1. $\lambda x.\, \lambda y.\, x\ (y\ x)$
2. $\lambda a.\, \lambda b.\, a\ (b\ a)$
3. $\lambda y.\, \lambda x.\, y\ (x\ y)$
4. $\lambda p.\, \lambda q.\, q\ (p\ q)$
5. $\lambda x.\, \lambda y.\, y\ (x\ y)$
6. $\lambda u.\, \lambda v.\, v\ (u\ v)$
7. $\lambda x.\, \lambda y.\, x\ (x\ y)$
8. $\lambda m.\, \lambda n.\, m\ (m\ n)$
9. $\lambda x.\, \lambda y.\, x\ (y\ z)$
10. $\lambda a.\, \lambda b.\, a\ (b\ z)$
11. $\lambda a.\, \lambda b.\, a\ (b\ w)$
12. $\lambda z.\, \lambda y.\, z\ (y\ z)$

(a) Group them into α-equivalence classes.

(b) Three of the terms above mention z. Two of those three are α-equivalent to each other and the third is not. Say which, and what makes the difference.

AI use declaration. I have not used AI tools when solving this problem.

Answer.

The classes are: *{1, 2, 3 and 12 }, {4, 5 and 6}, {7, 8}, {9, 10}, {11}.*

Justification for (b).

Terms 9 and 10 are α-equivalent but term 12 isn't α-equivalent to them because in terms 9 and 10, z is a free variable, while in term 12, z is bound by the outer abstraction λz.
α-equivalent is obtained by renaming bound variables. Free variables must therefore remain unchanged

Problem 1.2 (Call by name versus call by value) [No AI]

Reduce the term

$$(\lambda x.\, \lambda y.\, y)\ \big((\lambda f.\, f\ (f\ z))\ (\lambda w.\, w)\big)\ c$$

as far as it goes, twice: once under **call by value** and once under **call by name**. Write one β-reduction per line and underline the redex you contract. Then say how many steps each took, and what the longer one spent them on.

AI use declaration. I have not used AI tools when solving this problem.

Answer.

Call by value takes *5* steps, call by name *2* steps.

Derivation, call by value.

$(\lambda x.\lambda y.y)\ ((\lambda f.f(fz))(\lambda w.w))\ c$
$\to (\lambda x.\lambda y.y)\ ((\lambda w.w)\underline{((\lambda w.w)z)})\ c$
$\to (\lambda x.\lambda y.y)\ \underline{((\lambda w.w)z)}\ c$
$\to \underline{((\lambda x.\lambda y.y)z)}\ c$
$\to \underline{((\lambda y.y)c)}$
$\to c$
So, call by value $= 5\beta$*-reductions*

Derivation, call by name.

Let $A{=}((\lambda f.f(fz))(\lambda w.w))$
Then :
$\underline{((\lambda x.\lambda y.y)A)}c$
$\to \underline{((\lambda y.y)c)}$
$\to c$
So, call by name $= 2\beta$*-reductions*

What the longer one bought.

Call by value spent three extra steps evaluating the first argument, even though that argument is never used because x does not occur in the body λy.y. Call by name avoids this unnecessary work

Problem 1.3 (Church pairs) [No AI]

Recall the encodings from the lab:

$$\mathsf{tru} = \lambda t.\, \lambda f.\, t \quad \mathsf{fls} = \lambda t.\, \lambda f.\, f \quad \mathsf{pair} = \lambda f.\, \lambda s.\, \lambda b.\, b\ f\ s$$

$$\mathsf{fst} = \lambda p.\, p\ \mathsf{tru} \qquad \mathsf{snd} = \lambda p.\, p\ \mathsf{fls}$$

(a) Write a term swap for which swap (pair x y) reduces to pair y x. Here you may use the aliases.

(b) Write the term that extracts y from pair x (pair y z).

(c) Verify your answer to (b) by reduction, expanding every alias, so that no pair, fst, snd, tru or fls is left.

AI use declaration. I have not used AI tools when solving this problem.

Answer.

(a) $\mathsf{swap} = \lambda p.pair(snd\ p)(fst\ p)$

Answer.

(b) $fst(snd(pair\ x(pair\ y\ z)))$

Derivation for (c).

$(\lambda p.p(\lambda t.\lambda f.t))((\lambda p.p(\lambda t.\lambda f.f))((\lambda f.\lambda s.\lambda b.b\ f\ s)x((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)))$
$\to (\lambda p.p(\lambda t.\lambda f.f))((\lambda f.\lambda s.\lambda b.b\ f\ s)\ x\ ((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z))(\lambda t.\lambda f.t)$
$\to (\lambda f.\lambda s.\lambda b.b\ f\ s)\ x((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)(\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$
$\to (\lambda s.\lambda b.b\,x\,s)((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)(\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$
$\to (\lambda b.b\,x\,((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z))(\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$
$\to (\lambda t.\lambda f.f)\ x((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)(\lambda t.\lambda f.t)$
$\to (\lambda f.f)((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)(\lambda t.\lambda f.t)$
$\to ((\lambda f.\lambda s.\lambda b.b\ f\ s)\ y\ z)(\lambda t.\lambda f.t)$
$\to ((\lambda s.\lambda b.b\,y\,s)\ z)(\lambda t.\lambda f.t)$
$\to (\lambda b.b\,y\,z)(\lambda t.\lambda f.t)$
$\to (\lambda t.\lambda f.t)\ y\ z$
$\to (\lambda f.y)\ z$
$\to y$

Problem 1.4 (Testing for zero) [No AI]

A Church numeral applies its first argument to its second n times: $\mathsf{c}_n = \lambda s.\, \lambda z.\, s^n\, z$, where $s^0\, z$ is z, $s^1\, z$ is $s\ z$, $s^2\, z$ is $s\ (s\ z)$, and so on. The booleans are tru and fls as in problem 1.3.

(a) Write a term iszro such that $\mathsf{iszro}\ \mathsf{c}_0$ reduces to tru, and $\mathsf{iszro}\ \mathsf{c}_n$ reduces to fls for every $n > 0$. You may **not** use aliases: write the term out.

(b) Verify it on c_0 and on c_2 by full β-reduction, expanding every numeral and boolean. If a written-out term repeats, you may name it, provided you say what the name stands for.

(c) In one sentence, why does it work for *every* $n > 0$ and not only for the one you checked?

AI use declaration. I have not used AI tools when solving this problem.

Answer.

(a) $\mathsf{iszro} = \lambda n.n(\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$

Derivation for (b).

$(\lambda n.n(\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t))(\lambda s.\lambda z.z)$
$\to (\lambda s.\lambda z.z)(\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$
$\to (\lambda z.z)(\lambda t.\lambda f.t)$
$\to \lambda t.\lambda f.t$
Therefore, iszero $c_0 \to \lambda t.\lambda f.t$ *which is tru*
$(\lambda n.n(\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t))(\lambda s.\lambda z.s(sz))$
$\to (\lambda s.\lambda z.s(sz))(\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t)$
$\to (\lambda z.(\lambda x.\lambda t.\lambda f.f)((\lambda x.\lambda t.\lambda f.f)z))(\lambda t.\lambda f.t)$
$\to (\lambda x.\lambda t.\lambda f.f)((\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.t))$
$\to (\lambda x.\lambda t.\lambda f.f)(\lambda t.\lambda f.f)$
$\to \lambda t.\lambda f.f$
Therefore, iszero $c_2 \to \lambda t.\lambda f.f$ *which is fls*

Explanation for (c).

For every n>0, c_n *applies* $\lambda x.\lambda t.\lambda f.f$ *at least once, and this function ignores its argument and returns* $\lambda t.\lambda f.f$*, whereas* c_0 *applies it zero times and therefore returns the initial* $\lambda t.\lambda f.t$

Problem 1.5 (Inlining, audited)

A retry helper. It reports how many attempts fit in the time allowed.

```
timeout = 30                  # seconds we are willing to spend in total

def make_retry(limit):
    def retry(timeout):       # seconds to allow for one attempt
        spent = 0
        attempts = 0
        while spent < limit:
            attempts += 1
            spent += timeout
        return attempts
    return retry

run = make_retry(timeout)
print(run(10))
```

It prints `3`: three attempts of ten seconds fit inside thirty. *Inlining* `make_retry` means replacing `make_retry(timeout)` by its body, with the parameter substituted.

(a) **[AI allowed]** Ask an assistant to do that inlining, and record what it produces.

(b) **[No AI]** Run its answer. If the program no longer prints `3`, say what went wrong, in the language of the substitution rule. If it prints `3`, read what it changed and say how that avoided the trap. Either way the evidence is the code it wrote, not its account of what it did.

(c) **[No AI]** Give a correct inlining.

(d) **[No AI]** Write the corresponding λ-term, of the shape $(\lambda \mathit{limit}.\, \lambda \mathit{timeout}.\, \ldots)\ \mathit{timeout}$, substitute correctly, and name the step the naive inlining skipped.

AI use declaration. I have used AI (ChatGPT-5.5) in item (a) of this problem, in accordance with the course rules and the problem statement.

Answer.

(a) The transcript is submitted as `transcript.md`, and printed in the appendix.

Answer.

(b) Its program prints *3*, where the original prints `3`. The inlining is *correct*.

Justification for (b).

The assistant avoided variable capture by introducing the fresh variable _limit before the definition of retry. Thus _limit keeps the value of the outer timeout (30), while the parameter timeout of retry still receives 10. It therefore did not substitute the free timeout underneath the inner timeout binder, where that occurrence would have become captured

A correct inlining, for (c).

```python
timeout = 30

def retry(retry_timeout):
    spent = 0
    attempts = 0
    while spent < timeout:
        attempts += 1
        spent += retry_timeout
    return attempts

run = retry

print(run(10))
```

Derivation for (d).

$(\lambda limit.\lambda timeout.B(limit, timeout))timeout \equiv_\alpha (\lambda limit.\lambda t.\, B(limit, t))\ timeout$
$\rightarrow_\beta \lambda t.B(timeout, t)$

The skipped step is α-conversion (α-renaming) of the inner bound variable before capture-avoiding substitution

Appendix. My transcript for problem 1.5

This is `transcript.md`, printed as it stands. Hand that file in as well. The appendix is here so that a marker reading the PDF sees the exchange without opening anything else.

Copy every prompt and every reply into it, in the order they happened, and change nothing else. Chat interfaces have no *export transcript* button, so this is copying and pasting, one message at a time. If the copy loses the line breaks inside a code block, put them back and say so here.

````
# New chat

**User:** Anonymous
**Created:** 8/28/2026 12:50:36
**Updated:** 8/28/2026 12:50:47
**Exported:** 8/28/2026 12:50:56
**Link:** [https://chatgpt.com/?temporary-chat=true](https://chatgpt.com/?temporary-chat=true)

## Prompt:
8/28/2026, 12:50:34 PM

timeout = 30
def make_retry(limit):
            def retry(timeout):
                            spent = 0
                            attempts = 0
                             while spent < limit:
                                    attempts += 1
                                    spent += timeout
                             return attempts
          return retry
 run = make_retry(timeout)
print(run(10))

Inline the call to make_retryin this program: replacemake_retry(timeout)withthebodyofmake_retry,
↪  substituting its parameter for the argument. The behaviour must not change.

## Response:
8/28/2026, 12:50:43 PM

> **Встраивая замыкание**
>
> Я проверяю, как встроить вызов без изменения поведения: внутренний параметр `timeout` нужно сохранить, а
↪  внешнее значение `limit` - захватить заранее. Поэтому сначала присваиваю `limit = timeout`, затем
↪  определяю `retry` и использую его как `run`.
>
> **Встроил замыкание**
>
>
>
> Обработка заняла 7s

You need to avoid **variable capture** because both the outer argument and the inner function use the name
↪  `timeout`. A behavior-preserving inlining is:

```python
timeout = 30

_limit = timeout

def retry(timeout):
    spent = 0
    attempts = 0
    while spent < _limit:
        attempts += 1
        spent += timeout
    return attempts

run = retry

print(run(10))
```

Output:

```text
````
````

````
3
```

Here `_limit` preserves the original `30`, while the `timeout` parameter of `retry` receives `10`. Simply
↪  replacing `limit` with `timeout` inside `retry` would incorrectly make both refer to `10`, changing
↪  the result.



---
Powered by [ChatGPT Exporter](https://www.chatgptexporter.com)
````
````